

Senior Capstone Project README: Wearable Alert Device

University of Oklahoma
School of Electrical and Computer Engineering
Semester: Spring 2026

Team Members:

Project Manager - Jonas Wollman
Hardware Engineer - Ryan Kiesau
Hardware Engineer - Cody Chan
Software Engineer - Alex Ho

README written by Alex Ho

Overview

This repository contains all project files pertaining to the Wearable Alert Device project. The following information provides guidance for setting up the software development environment as well as a general description of each of the major components of the software.

Table of Contents

1. [Downloading Arduino IDE](#)
2. [Initial Environment Setup](#)
 - [Arduino Nano ESP32 Board Package Installation](#)
 - [Pin Numbering Setting Configuration](#)
 - [Required Library Dependencies](#)
 - [Verifying and Uploading Code to a Microcontroller](#)
3. [Software Development](#)
 - [WearableAlertDevice.ino](#)
 - [config.h](#)
 - [WirelessDataTransmission.h/.cpp](#)
 - [GunshotDetection.h/.cpp](#)

Downloading Arduino IDE

For this Capstone project, the Arduino IDE was used as the software development environment. The Arduino IDE can be downloaded [here](#). Ensure that some version of Arduino IDE 2.0 is downloaded, not the legacy Arduino IDE 1.0.

Back to [Table of Contents](#)

Initial Environment Setup

Once the Arduino IDE has been installed, some initial setup in its environment is needed before code can be correctly uploaded into the Arduino Nano ESP32 microcontroller. There are three important parts to the setup: installing the Arduino Nano ESP32 board package, setting the pin numbering to GPIO numbers, and installing library dependencies.

Back to [Table of Contents](#)

Arduino Nano ESP32 Board Package Installation

The first stage of the environment setup is installing the proper board package so that the environment can correctly compile code and upload programs for Arduino Nano ESP32 microcontrollers. In the Arduino IDE, look on the left for the Boards Manager tab. Click on the Boards Manager tab to open it up.

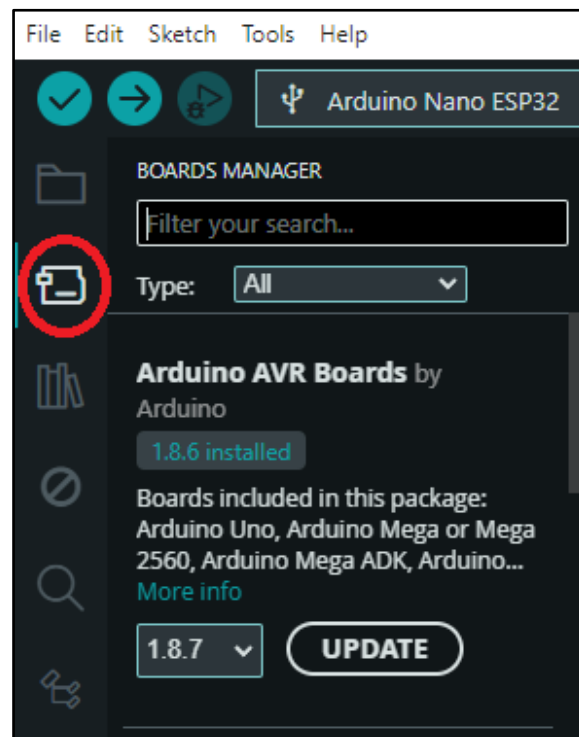


Figure 1: Screenshot of Boards Manager tab in the Arduino IDE (circled in red).

In the search bar, type "Arduino ESP32 Boards." This package will contain the settings to recognize Arduino Nano ESP32 hardware. Select the Install button if the package is not already installed.

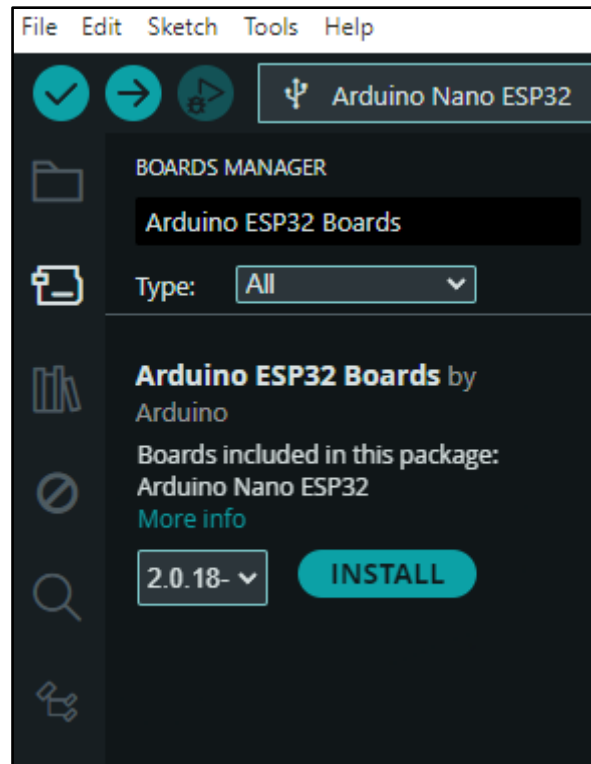


Figure 2: Screenshot of the Arduino ESP32 Boards package not installed yet.

Once the package has been installed, the Arduino IDE will automatically install all the common software libraries associated with ESP32 microcontrollers, such as WiFi, Bluetooth, etc. No additional steps are needed to install these libraries.

Back to [Table of Contents](#)

Pin Numbering Setting Configuration

The software developed for this project has been written specifically to use the Arduino Nano ESP32's GPIO numbers, not the standard Arduino pin numbers that are labeled on the physical device. **NOTE: IF THE PIN NUMBERING IS SET TO ARDUINO PIN NUMBERS INSTEAD OF GPIO NUMBERS, THERE WILL BE A GOOD CHANCE THAT INCORRECT PINS ON THE MICROCONTROLLER WILL BE USED BY THE SOFTWARE, WHICH WILL MOST LIKELY PREVENT THE HARDWARE FROM FUNCTIONING PROPERLY.**

To change the pin numbering setting in the Arduino IDE, navigate to `Tools >> Pin Numbering` at the ribbon near the top of the IDE. In the Pin Numbering setting, there will be two options: `By Arduino pin (default)` and `By GPIO number`

(legacy). Set the Pin Numbering setting to the By GPIO number (legacy) option.

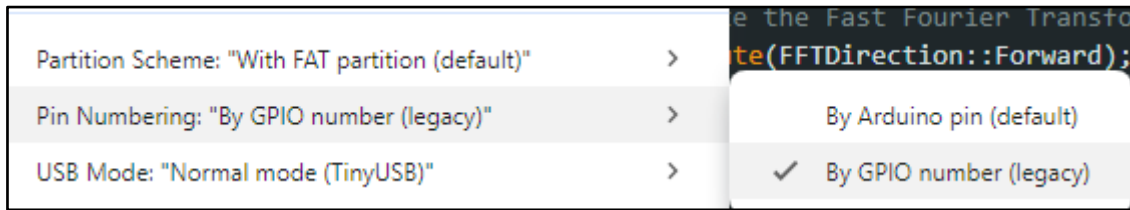


Figure 3: Screenshot of Pin Numbering setting being set to the "By GPIO number (legacy)" option.

The Pin Numbering setting can be found in the Tools tab in the ribbon near the top of the IDE.

Back to [Table of Contents](#)

Required Library Dependencies

There are a few open-source libraries that the software relies on for its functionality. These external libraries are not automatically installed by the Arduino IDE and will have to be installed manually. Below is a list of the open-source libraries:

1. [DFRobot_GNSS](#) by DFRobot
2. [arduinoFFT](#) by kosme

The libraries can be downloaded by using the GitHub links above, or they can also be found in the `Software/dependencies` folder for this project. Once these libraries are downloaded, they need to be installed manually into the Arduino IDE. Navigate to `Sketch >> Include Library >> Add .ZIP Library...` at the ribbon near the top of the IDE. After clicking the `Add .ZIP Library` option, find the local `.zip` files of the libraries and add them to the Arduino IDE.

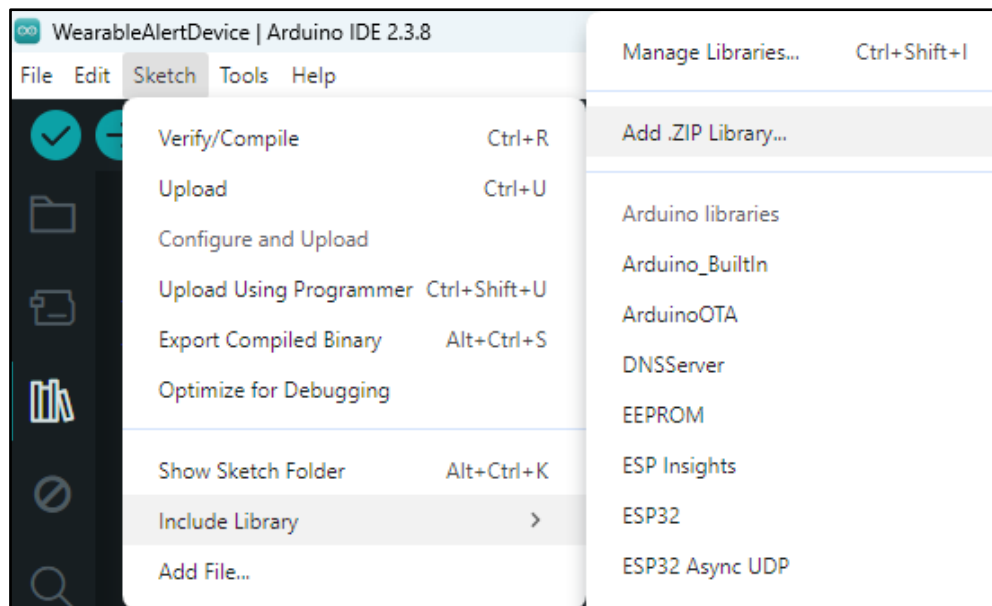


Figure 4: Screenshot of the navigation to the "Add .ZIP Library..." option in the Arduino IDE.

Back to [Table of Contents](#)

Verifying and Uploading Code to a Microcontroller

When connecting an Arduino Nano ESP32 microcontroller to the computer for the first time, it is possible that the Arduino IDE does not recognize the hardware as an Arduino Nano ESP32 model specifically. Before verifying or uploading code to the microcontroller, first navigate to `Tools >> Board`. Ensure that the board type says "Arduino Nano ESP32." If the board type does not say that, set the board type to "Arduino Nano ESP32" by navigating to `Tools >> Board >> Arduino ESP32 Boards` and selecting "Arduino Nano ESP32."

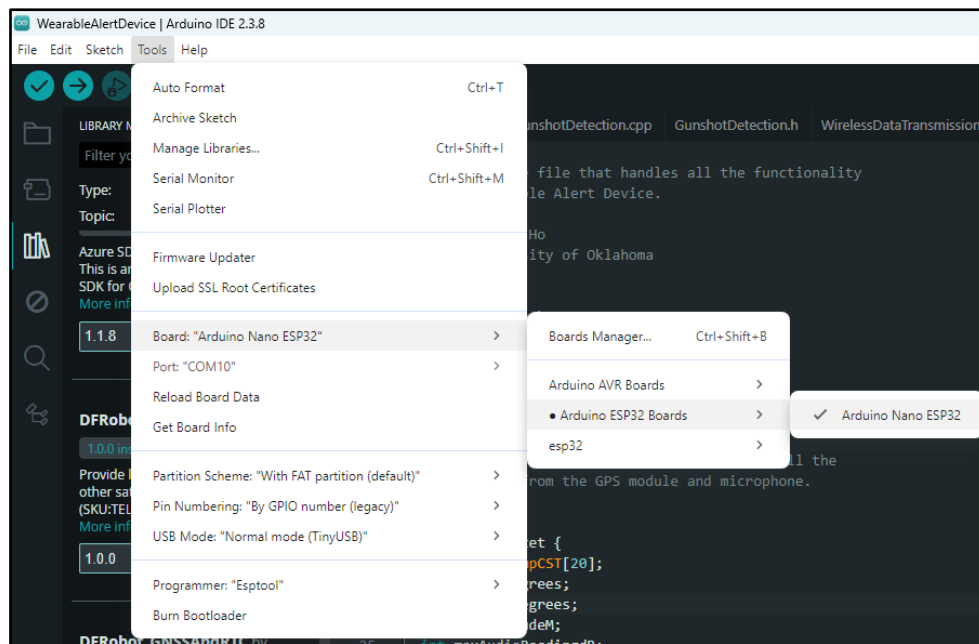


Figure 5: Screenshot of the IDE's board type being set to "Arduino Nano ESP32."

To verify that the Arduino IDE properly recognizes the microcontroller as an Arduino Nano ESP32, connect the microcontroller to the computer's USB-A port, and the IDE should display the COM port number and "Arduino Nano ESP32."

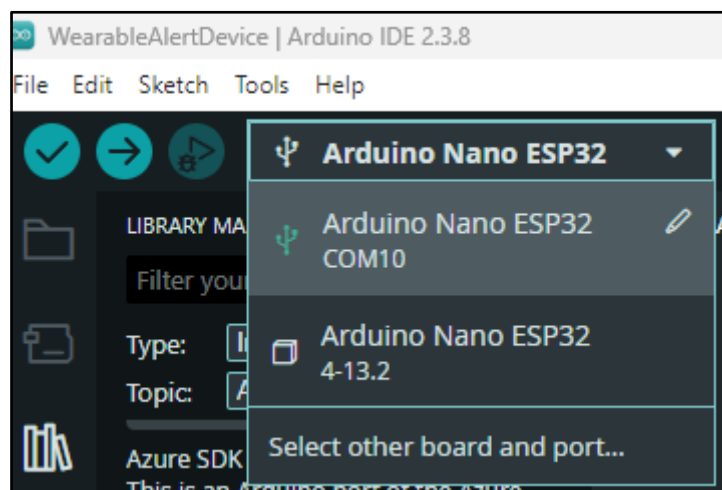


Figure 6: Example screenshot of the IDE detecting an Arduino Nano ESP32 on COM Port 10.

Once the IDE properly recognizes the Arduino Nano ESP32, code can now be properly installed on the hardware. There are two important buttons in the IDE for installing software onto the microcontroller: the `Verify` button and `Upload` button.

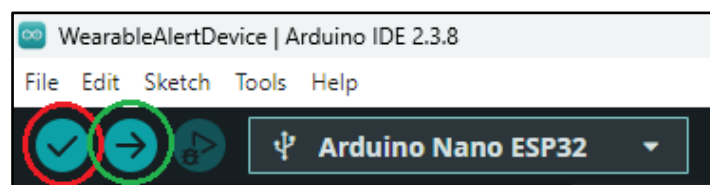


Figure 7: Screenshot of the `Verify` button (circled in red) and `Upload` button (circled in green) in the Arduino IDE.

The `Verify` button, identified by a checkmark symbol, compiles the code and ensures there are no compile errors. This button does not upload the software into the microcontroller. The `Upload` button, identified by an arrow pointing right, performs the same function as the `Verify` button. However, once the code has been successfully compiled, the `Upload` button will then upload the software to the connected microcontroller. **For best practices, it is best to first press the `Verify` button before pressing `Upload` button when uploading new code to the microcontroller.**

Back to [Table of Contents](#)

Software Development

The following section provides a general description of each of the software files. If further details are needed, comments and documentation can be found within the software files themselves.

Back to [Table of Contents](#)

WearableAlertDevice.ino

`WearableAlertDevice.ino` is the main software file for this project. This file serves as the device's main source for functionality. The Arduino Nano ESP32 will look to this file for how to control the connected hardware. Functions and variables from supporting files such as `config.h`, `WirelessDataTransmission.h`, and `GunshotDetection.h` can also be used by `WearableAlertDevice.ino`.

Back to [Table of Contents](#)

config.h

`config.h` contains all the adjustable settings for the Wearable Alert Device. Instead of going into each of the different software files to change certain settings, which can introduce potential bugs if not carefully done, these settings can be changed directly in `config.h`, and those changes will automatically propagate throughout the rest of software. As a result, changing a specific setting for the Wearable Alert Device only requires changing a single value in `config.h`, and developers do not need to parse through all the complex code to change each instance of the setting that appears in the code.

Back to [Table of Contents](#)

WirelessDataTransmission.h/.cpp

`WirelessDataTransmission.h` handles the software that wirelessly transmits the data from the Wearable Alert Device to a mobile app. For this project, Bluetooth Low-Energy was used as the protocol for transmitting data wirelessly. Thus, `WirelessDataTransmission.h` contains the software that initializes the microcontroller's Bluetooth Low-Energy module and handles when a mobile device connects or disconnects from the Wearable Alert Device.

Back to [Table of Contents](#)

GunshotDetection.h/.cpp

`GunshotDetection.h` handles the audio sampling and processing needed to detect a gunshot. This file creates all the necessary buffers that store audio samples collected by the microphone. In addition, the file also performs the mathematical conversions needed to convert the analog voltage from the microphone into a fairly accurate dB Sound Pressure Level reading. What is important to note about `GunshotDetection.h` is that it includes the previously-mentioned `arduinoFFT` library, allowing `WearableAlertDevice.ino` to use `arduinoFFT`'s frequency conversion and analysis functions.

Back to [Table of Contents](#)